

Universidad Carlos III de Madrid



B.S. in Telecommunication Technologies Engineering

**MACHINE LEARNING CLASSIFIER FOR FAULT
DIAGNOSIS IN ROTARY MACHINES**

AUTHOR:

Víctor Manuel Cabaleiro Valado

Table of Contents

1.0	Abstract.....	4
2.0	Problem.....	4
3.0	Objective	5
4.0	Background	5
4.1	Key Concepts	5
4.2	Related Work	6
5.0	Design	7
5.1	Use Cases	7
5.2	Requirements	8
5.2.1	Functional Requirements	8
5.2.2	Interface Requirements	9
5.2.3	Performance Requirements.....	11
5.3	High-Level Architecture	12
5.4	Detailed Architecture	14
5.5	Data Management and Storage	17
6.0	Characterization of Vibration Fault	18
6.1	Failure Vibration Modes In Rotating Machinery.....	18
6.2	Fault Detection Techniques And Algorithms	20
6.3	Metrics and Evaluation for Fault Classification	24
6.4	Preprocessing And Feature Extraction Results.....	25
6.5	Machine Learning Models	26
6.5.1	Fault Detection Performance	26
6.5.2	Fault Categorization Performance	27
7.0	Development Plan	28
7.1	Tasks	28
7.2	Schedule.....	28
7.3	Deliverables	30
8.0	Key Personnel.....	30
9.0	Facilities and Equipment.....	31
10.0	References	31

Acknowledgements

I would like to express my deepest gratitude to everyone who has supported me throughout the development of this Final Degree Project.

First and foremost, I am grateful to Universidad Carlos III de Madrid for providing me with the opportunity to participate in a non-European mobility program and for the academic support received throughout my studies.

I would especially like to thank Dr. David C. Jensen from the University of Arkansas for his guidance, dedication, and support during the development of this project. His expertise and encouragement were essential to the progress and completion of this work.

I also want to thank the team at the ENRC lab for granting me access to the equipment and for their assistance during the data collection and analysis process.

On a more personal note, I am deeply thankful to my mother for her constant daily effort to always give me the very best, and to my father for his absolute trust and unwavering support. I truly could not have asked for better parents to accompany me through this journey. To my grandparents, for being a steady and generous presence in my life. To Tobias, my childhood dog, whose memory remains forever in my heart. And to Pepa, who has shared with me a wonderful chapter of my life that now comes to an end.

1.0 Abstract

Rotary machines are a vital part of industrial machinery and are used across many different industries. However, these machines are naturally subject to faulty parts due to age. These faults must currently be checked through scheduled maintenance, but to better combat these issues, engineers need a way to collect data and better predict faulty parts [4]. Machine Learning (ML) has become an increasingly popular solution to the problem of collecting data and classifying machine faults. We want to design a machine learning program that will be able to use different algorithms to improve the classification and storage of fault diagnosis data.

There are many different types of classifiers that are useful for different tasks. Our goal is to use these classifiers, put them in one program, and allow engineers to easily collect and analyze data from these classifiers to diagnose machine faults. Machine learning can change the way people collect and analyze data; it can decrease the amount of time and resources dedicated to completing a specific task — this is the case for fault diagnosis. The integration of machine learning models can be complex and difficult, but the ability to predict and identify machine faults quickly and efficiently is essential to the development of machine fault diagnosis.

2.0 Problem

Rotating machines are widely used in many industries, such as manufacturing and energy. However, these machines are prone to developing faults over time, which can lead to unexpected breakdowns. This poses a serious problem because, without proper fault detection and monitoring, companies can face costly downtime, expensive repairs, and even safety risks. These machines often operate continuously, making any sudden failure a significant issue that disrupts production and leads to financial losses. Therefore, the need to monitor machine health and detect faults before they escalate is critical.

Current maintenance approaches, like scheduled maintenance, replace parts based on a predetermined schedule rather than their actual condition. This method can result in two undesirable outcomes: either the machine fails before the maintenance occurs, leading to unplanned downtime, or the parts are replaced prematurely, wasting resources and money. Both scenarios highlight the inefficiency of not having a real-time, condition-based monitoring system. This lack of a solution means that industries remain vulnerable to operational disruptions and increased maintenance costs.

The absence of effective fault detection can also impact the safety of workers. Sudden machine failures can cause accidents, making it important to have a system that detects and predicts these issues in advance. Furthermore, as machines become more complex, identifying and diagnosing faults becomes more challenging. This emphasizes the importance of developing an automated system capable of continuously monitoring machines and predicting potential failures, ensuring not only efficiency but also workplace safety.

3.0 Objective

The objective of this project is to develop a machine learning-based tool for real-time fault diagnosis in rotary machines, which are critical components in many industries. The system aims to leverage various machine learning classifiers to analyze sensor data from these machines and identify potential faults before they lead to major failures, minimizing downtime and improving operational efficiency [7].

The tool will collect and process real-time vibration data from accelerometers attached to rotary machines. This data will then be used to train and apply multiple machine learning classifiers, enabling the system to detect and classify faults such as bearing failures, misalignments, and other issues [7]. By utilizing a range of classifiers, the system will ensure robust and accurate fault detection that is adaptable to different machine conditions and fault types.

Additionally, the project aims to provide a user-friendly interface that allows engineers to interact with the system, select classifiers, modify feature extraction methods, and train the tool using historical data. This flexibility will make it easier for users to fine-tune the system for their specific needs, ensuring a tailored approach to fault diagnosis.

The overall goal is to create a platform that offers real-time, accurate, and scalable fault detection, reducing maintenance costs, improving machine reliability, and enhancing the ability of industries to monitor the health of their equipment. The system's integration of multiple machine learning classifiers will push the boundaries of current predictive maintenance technologies, providing a cutting-edge solution for rotary machine health monitoring.

4.0 Background

4.1 Key Concepts

The Machine Fault Simulator was created by SpectraQuest, Inc. and its main purpose is to gain an understanding of different vibration signatures for industrial machine health diagnosis [1]. The simulation machine can mimic the conditions of real-world machine faults without affecting real-world production [1]. Machine diagnosticians can use the machine to gain expertise and experience in diagnosing these different faults and gain a deeper understanding of real-world vibration spectra [1]. The Machine Fault Simulator can also collect data on bearings that cause faults and those that don't. For our Machine Classifiers, we can collect data from the Machine Fault Simulator from multiple scenarios of single and multi-fault rotary machines. We can use this data to train and test Machine Learning algorithms for rotary machine fault diagnosis. Data is collected from the fault simulator by switching out nominal components on the fault simulator with one or more faulty components [5]. The machine is then turned on and set to a specific rpm value [5]. Data is then collected for 10 seconds by the sensors once the target speed has been reached [5]. The process is repeated for each faulty component that is being tested [5].

Machine Learning is a field of computer science that uses data and statistical analysis to draw inferences from patterns in data [2]. Machine learning is a vast field with multiple different

algorithms that can be used to learn from and classify data. Machine learning models are divided into three different methods of training data: Supervised, Unsupervised, and Semi-supervised learning [3]. Supervised learning trains algorithms on labeled datasets to predict outcomes by changing the weights of the model to produce the most accurate outcome [3]. Unsupervised learning trains algorithms on unlabeled datasets to analyze and cluster data to find new patterns and information from the datasets [3]. Semi-supervised learning is a combination of supervised and unsupervised learning that “uses smaller labeled datasets to guide classification and feature extraction from a larger, unlabeled dataset” [3]. For each of these methods, programmers can use different Machine Learning algorithms to help with classification. Some of the most popular algorithms include Decision Trees, Neural Networks, Naïve Bayes, Clustering, K-means, SVM, K-nearest neighbor, Logistic Regression, and Linear Regression. These different machine learning algorithms can be used for specific tasks that best analyze the data for classification and prediction. For example, neural networks are great at recognizing patterns and are used in fields such as natural language translation and speech recognition [3]. Once the data has been trained and tested on machine learning algorithms, programmers must assess their model’s performance. Different assessments can be used to determine the performance of a machine learning model, including the confusion matrix, accuracy, precision, recall, and F1 scores.

4.2 Related Work

Machine learning in machine health diagnostics has become a growing research area. Machine learning algorithms would provide a better maintenance strategy for industrial machinery and improve machine lifetime and plant profits by continuously checking for machine faults rather than checking manually. A research paper by Saha et al. investigated using machine learning to diagnose bearing faults using an SVM algorithm. The paper investigated four common bearing faults: ball fault, outer race fault, inner race fault, and cage fault [4]. The methodology includes data generation, signal conditioning, feature extraction, model development, and model prediction [4]. The data generation stage generated the vibration data using an accelerometer sensor mounted on the bearing housing [4]. The signal conditioning stage changed the data into a .csv or .txt data for the software to read [4]. The feature extraction stage decided which features from the dataset would be used to train and test the model [4]. These include time-domain features like kurtosis: the tailedness of a distribution of variables, crest factor: the difference between the peak and average of a signal in decibels, and form factors: the specification of components in relation to their design [4]. The model development stage decided the approach for fault detection using a machine learning algorithm [4]. Finally, the model prediction stage analyzed the results from the classification model using accuracy, precision, recall, and F1 score [4]. The developers used the support vector machine as their machine learning approach by finding “the optimal hyperplane that differentiates the training data into two binary classes by maximizing the margin” [4]. The developers also used Particle Swarm Optimization to determine a variable that “optimizes the parameters based on function $f(x)$, where $f(x)$ is a fitness function or objective function” [4]. After training and testing the SVM and PSO-SVM algorithms, a confusion matrix was used to calculate accuracy, precision, recall, and F1 score [4]. Overall, the SVM performed well with 92% accuracy, with an improvement of almost 2% using the PSO-SVM model [4].

Currently, this problem has been tackled before by the Mechanical Engineering department at the University of Arkansas. The datasets used for this experiment tested 38 different fault scenarios at

three different RPMs with 25 datasets collected for each test [6]. The features were then extracted from the datasets and split into 80% training and 20% testing data [6]. There were five different machine learning algorithms used on the testing data including K-nearest neighbor, Gaussian Naïve Bayes classifier, Support Vector Machine, Decision Tree, and Random Forest [6]. The results were measured based on how accurate the classifications for each algorithm were on the test set [6]. The accuracy was measured based on partially correct outcomes as well as correct outcomes [6]. This weighs a classification that might be partially correct higher than one that is completely wrong, so if the model classifies the correct location but the wrong fault it will give partial accuracy to the model [6]. The KNN algorithm had an accuracy of 77.1794%, the SVM machine had an accuracy of 82.0512%, the Gaussian Naïve Bayes algorithm had an accuracy of 93.0769%, The Decision Tree algorithm had an accuracy of 98.4515%, and finally, the Random Forest algorithm had an accuracy of 99.979% [6]. The Random Forest resulted in the highest accuracy score and was used to train and test the datasets of three different frequency levels for each fault [6]. They were initially tested individually on 25Hz, 50Hz, and 75Hz with high accuracy scores. They were then trained on training data of 25 Hz and tested on 50 Hz which did not perform well, with an accuracy of 22.0513% [6]. The training data was then split evenly between 50 Hz and 25 Hz datasets as well as split into thirds for 25 Hz, 50 Hz, and 75 Hz. When trained on different datasets the accuracy improved to above 99% for each frequency level [6].

Our implementation will be similar to other machine learning algorithm implementations used to determine machine faults. The biggest difference in our implementation is the datasets that need to be processed. Other projects mainly focus on one type of mechanical fault. Saha et al. focused on bearing faults specifically, but there are other types of faults it does not take into consideration or look at such as gear pitting faults [4]. Our project focuses on both multiple different single-fault and multi-fault cases. This allows the machine learning algorithm to determine more faults as well as combinations of faults with more accuracy than a machine learning algorithm that is only applicable to one specific type of fault. To improve upon the original design of this project, we would like to train and test regression models to investigate classifying data that the model has and hasn't been trained on. This would give us a higher level of accuracy over a wider range of fault simulations, therefore developing machine learning algorithms that would better adapt to determining different kinds of faults with better accuracy and quickness.

5.0 Design

5.1 Use Cases

The users of our program are Dr. Jensen and students of the MEEG department with access to Dr. Jensen's lab at the ENRC. They are able to do the following with our design:

- Run the program with little to no knowledge of ML/classifiers
- Import old and new data into the program from a CSV file/s
- Perform feature extraction on the data in the uploaded CSV file/s using statistical analysis so that the data can be turned into meaningful features which then can be classified

- Choose which model/s (SVM, Decision Tree, Linear Regression, Logistic Regression, or Neural Network) the user would like to use to classify the data
- Perform analysis through accuracy, precision, recall, F1-score, and a confusion matrix to determine the success of each ML model used
- Export the data they used along with the results from the classifiers and analysis of the classifiers to a CSV file

Our program is user friendly to faculty and students from the MEEG department and can be used alongside the Machine Fault Simulator. Our program uses prompts and friendly user interfaces to make it easily accessible for users without prior knowledge of machine learning or programming. So, the user can run the program and easily download data, perform feature extraction, run machine learning models, and print performance metrics quickly and efficiently without a background in machine learning.

5.2 Requirements

5.2.1 Functional Requirements

Our program is designed to analyze sensor data to detect and categorize faults in rotary machines. The process begins with data ingestion and preprocessing, where raw data from sensors measuring vibrations and rotational speed is collected. These measurements are stored in structured files, capturing frequency-domain and time-domain data. The frequency-domain data represents patterns in vibration signals over time, while the time-domain data records direct signal fluctuations. This combination provides a comprehensive view of the machine's condition. Before further analysis, the system ensures that all data is cleaned, formatted correctly, and free of inconsistencies. Noise filtering and missing data handling improve the accuracy of subsequent steps, resulting in a well-structured dataset ready for processing.

Next, the system applies feature extraction to transform raw sensor data into meaningful indicators. Time-domain analysis helps identify sudden changes in vibrations by extracting values such as peak amplitude, mean, standard deviation, and variance. On the other hand, frequency-domain analysis converts time-based signals into spectral components, revealing periodic faults that may not be obvious in direct readings. By combining both approaches, the system ensures that even subtle faults can be detected. These extracted features serve as input for machine learning models, allowing them to differentiate between normal and faulty operating conditions with greater accuracy.

After extracting the most relevant features, the system performs feature selection to refine the dataset further. By prioritizing essential attributes, the system reduces data complexity while maintaining classification accuracy. This eliminates unnecessary information, ensuring that machine learning models focus only on the most significant indicators. The final dataset consists of the most critical features necessary for detecting and categorizing faults, enhancing efficiency and reliability. With the refined dataset, the system proceeds to data splitting, where the information is divided into training and testing subsets. Typically, 80% of the data is allocated for training, while 20% is reserved for testing. This division ensures that the machine learning models have sufficient examples to learn from while also allowing for the evaluation of unseen data.

Proper data splitting is crucial for preventing overfitting and ensuring that the trained models can generalize well to new machine conditions.

Once the data is prepared, the system enters the model training phase. Various machine learning models are trained using the structured dataset, allowing them to recognize patterns in vibration and rotational speed readings. The models analyze historical data to learn how different faults manifest in sensor measurements. By fine-tuning parameters and optimizing classification techniques, the system ensures that the models achieve high accuracy in distinguishing normal operations from fault conditions. Following training, the models are applied to fault detection, determining whether a machine is operating normally or exhibiting a fault. The output is a simple binary classification: either a fault is detected, or the machine is functioning as expected. This step plays a crucial role in preventive maintenance, enabling early identification of issues before they escalate into serious failures. By continuously monitoring machine conditions, the system helps minimize downtime and optimize maintenance schedules.

Beyond basic fault detection, the system also performs fault categorization, assigning detected faults to specific types. This classification step provides deeper insights into the root causes of issues, indicating whether the fault originates from the bearings, shaft, or other machine components. By labeling faults into predefined categories, maintenance teams can take precise corrective actions tailored to the specific issue, improving overall efficiency in machine diagnostics and repairs. To validate the accuracy and reliability of the system, a performance evaluation is conducted. The effectiveness of each classification model is measured using key performance metrics, including accuracy, precision, recall, and F1-score. These metrics provide insight into how well the system differentiates between faulty and non-faulty conditions. Additionally, confusion matrices are generated to highlight areas where classification errors occur, helping to refine and improve the models over time. For models handling continuous predictions, additional evaluation metrics such as mean squared error and R^2 score are applied. Regular performance assessments ensure that the system consistently meets accuracy standards and adapts to new machine conditions effectively.

To assist users in interpreting results, the system includes data visualization capabilities. Graphical representations such as confusion matrices make it easier to understand fault patterns. These visual tools allow users to track machine health over time, detect potential failure trends, and make informed decisions about maintenance actions. The intuitive interface ensures that users with little to no background in machine learning can interact with the system effectively.

By integrating advanced machine learning techniques with structured data analysis, our system provides an effective and user-friendly solution for fault detection and categorization. Through real-time monitoring, accurate classification, and detailed reporting, it enhances machine reliability, reduces maintenance costs, and contributes to more efficient industrial operations.

5.2.2 Interface Requirements

The interface of this project involves a comprehensive approach to integrating hardware and software components to enable accurate and efficient fault detection in rotary machines. The

architecture is divided into four main components: data acquisition, feature extraction, machine learning model implementation, and user interaction through a menu-driven interface.

The hardware setup involves connecting accelerometer sensors to a rotary fault simulator to capture vibration data under various operational conditions. These sensors interface with a data acquisition system that converts analog signals into digital format and timestamps the data with RPM measurements. The data is stored in CSV files, ensuring compatibility with subsequent software processes.

The software component for feature extraction is implemented using Python, leveraging powerful data manipulation and analysis libraries such as Pandas, NumPy, and SciPy. The feature extraction process is designed to handle large datasets efficiently, processing data from 25, 50, and 75 RPM trials independently to avoid memory overload. The Frequency Domain feature extraction utilizes the Fast Fourier Transform (FFT) to convert time-series data into the frequency domain, allowing the calculation of metrics such as maximum frequency, mean frequency, variance, standard deviation, signal power, skewness, and kurtosis. This is achieved through a custom Python script that reads CSV files from the rotary machine fault simulator, applies FFT to each sensor's data, computes the statistical metrics, and saves the extracted features into new CSV files. In the Time Domain feature extraction, the script calculates statistical metrics directly from the raw sensor data, including mean, variance, standard deviation, root mean square (RMS), peak-to-peak values, skewness, and kurtosis. Both feature extraction methods are designed to systematically process all input files, outputting structured datasets ready for machine learning models.

The machine learning component is implemented using the scikit-learn library, incorporating a variety of classifiers such as Decision Tree, Support Vector Machine (SVM), Logistic Regression, Linear Regression, and Neural Networks. These models are utilized for both fault detection (binary classification of fault vs. no-fault) and fault categorization (multiclass classification of specific fault types). The machine learning pipeline involves splitting the data into training and testing sets, standardizing the data (where applicable), and training each model on both Frequency Domain and Time Domain features. Performance metrics, including accuracy, precision, recall, F1-score, confusion matrices, mean squared error, and R2 score, are automatically generated for each model and displayed to the user. The system is designed to run all classifiers in parallel, providing a comprehensive overview of model performance without requiring manual selection or input from the user. This approach ensures that the output is both thorough and easy to interpret, facilitating the identification of the most effective machine learning model for different fault detection scenarios.

In addition to the core functionality, the system includes a user-friendly menu interface that allows the user to select various options for dataset processing and model evaluation. Upon launching the program, the user is prompted to choose between different RPM datasets (25, 50, or 75 RPM), and the type of feature extraction they wish to perform (Frequency Domain or Time Domain). The menu further allows the user to select which machine learning model they would like to apply, with options for Linear Regression, Logistic Regression, Decision Tree, SVM, and Neural Networks. The user can also specify the range of data files they want to process, making it easier to handle large datasets. This menu-driven interface ensures that users can navigate through the entire fault detection pipeline effortlessly, from data loading to model evaluation, providing flexibility and ease of use.

The system interfaces seamlessly between hardware and software. Data collected from the hardware is formatted for software processing, and the machine learning models interact directly with the feature extraction output. The modular design of the software allows for easy updates and maintenance, with distinct modules handling data acquisition, preprocessing, model training, evaluation, and user interaction. The integration of SMOTE for handling imbalanced data and bandpass filtering for noise reduction further enhances the robustness and accuracy of the system, ensuring that the results are reliable and suitable for real-world applications.

5.2.3 Performance Requirements

For feature extraction, our program is handling three different folders, testing 25, 50, and 75 RPM machine fault trial data, with 38 different fault cases each having 25 trials per fault. This results in 950 CSV files per folder. Therefore it is important that our feature extraction program can handle this amount of data without crashing. Handling all three folders simultaneously resulted in too much memory use and crashed our program. To minimize memory usage, we decided to run one DataFrame at a time, output the csv file, and then empty the data frame so the memory would not be overloaded. Our plan currently is to create a while loop as a part of our menu function to handle one folder at a time, perform feature extraction on the folder, output the CSV file, and then empty the DataFrame to be used for another folder.

The large amounts of data can also cause a long runtime for our machine learning programs. Previously, we planned on running our feature extraction in the same program as our machine learning models, however, this resulted in a run time of up to 20 minutes, which was too long and unrealistic for our real-time scenarios. Preferably, our program would run in less than seven to ten minutes. Our solution to this problem is to run our feature extraction and models in separate programs. Our feature extraction is performed and then the data is placed into a CSV file that can be uploaded into the program for training and testing our machine learning models. This saves time by only running the feature extraction once, which takes seven to ten minutes per folder. So, as long as there are no updates to the folder, the feature extraction CSV files will remain the same and won't have to be run multiple times. This saves minutes of runtime on our program as the program for our machine learning models only takes about one minute to compute and print its results. As long as there is no need to update the feature extraction CSV files, this saves almost six minutes on run time, which is important for detecting faults in real-time scenarios.

To measure the results of our machine learning algorithms, we are looking at four performance metrics: accuracy, precision, recall, and F1-score. Additionally, we are printing the confusion matrix for each of our models, except for the linear regression model because it is a regression model rather than a classification one. Each model type has twelve different models to run, all made to test different data. We have one for each RPM speed (three total) and two for each feature extraction set. This results in 6 different tests for our model. Additionally, we are looking at the results for both the fault category and fault detection. Fault category refers to the specific fault type that the models will classify such as "Shaft Fault" or "Outer Bearing Fault" while fault detection refers to if there is any fault in the machine, but does not specify the type. This results in 12 performance metric outputs for each machine learning model (60 in total). Each algorithm's performance metrics are compared to determine which machine learning algorithms are best suited

for classifying different machine faults. Additionally, we are comparing which feature extraction values are best for determining the fault types and if there is a difference in the results for determining specific faults vs. detecting faults in the machine, but not specifying what kind. This will be used as a research tool, so while high-performance scores would be nice and useful, there are no specific performance requirements, rather it is important to test multiple different scenarios for fault detection to research which models are the most accurate, useful and can be applied in real-world scenarios. In short, the process of testing multiple different machine learning models as a way to research which methods are best for fault detection is the main goal of our project. Therefore high performance metrics are not as important as gaining information on what methods are best for fault detection, which methods do or don't work, and why.

Our final results were written in our statistical analysis paper, which documented the performance of each machine learning model as well as the feature extraction methods. We found through the performance metrics that overall, time-domain feature extraction was more successful than frequency-domain feature extraction. Time domain features had a range of 95-100% for fault detection with the most successful models being Decision Tree and Linear Regression with 100% accuracy and R2 score of 1.0. These models were able to detect the non-fault cases which other models had trouble predicting because the non-fault class had much smaller case numbers. Time domain features had a range of 8-100% for fault categorization with the most successful models being Decision Tree and Linear Regression with 96-98% for Decision Tree and a R2 score of 1.0 for the Linear Regression. The Frequency Features were more successful for the fault detection case with 96% for the SVM, Neural Network, and Logistic Regression models. The fault categorization performance was far lower with 1-8% accuracy with the best model being the Linear Regression model with an R2-score of 0.08. Detection of faults was much more successful than classification of faults in all cases. This is due to the smaller number of classes that must be sorted. For detection, there are only two classes, “faulty” and “not faulty” whereas classification has 38 classes, which is much harder for models to sort. The main issue with the detection code is that it has a small number of non-faulty cases when compared to faulty cases, but this can be remedied by adding more test cases. However, there were still models that were able to detect the faults with 100% accuracy even with this restriction. Overall, fault detection models that seemed to perform better consisted of Neural Networks, Decision Trees, Logistic Regression, and Support Vector Machines with consistently high scores. For fault categorization, models such as Logistic Regression, Decision Trees, and Neural Networks seemed to perform better while the Logistic Regression and SVM models failed to predict fault cases in many scenarios.

5.3 High-Level Architecture

The design of this project involves a comprehensive approach to integrating hardware and software components to enable accurate and efficient fault detection in rotary machines. The architecture is divided into three main components: data acquisition, feature extraction, and machine learning model implementation.

The hardware setup involves connecting accelerometer sensors to a rotary fault simulator to capture vibration data under various operational conditions. These sensors interface with a data acquisition system that converts analog signals into digital format and timestamps the data with

RPM measurements. The data is stored in CSV files, ensuring compatibility with subsequent software processes.

The software component for feature extraction is implemented using Python, leveraging powerful data manipulation and analysis libraries such as Pandas, NumPy, and SciPy. The feature extraction process is designed to handle large datasets efficiently, processing data from 25, 50, and 75 RPM trials independently to avoid memory overload. The Frequency Domain feature extraction utilizes the Fast Fourier Transform (FFT) to convert time-series data into the frequency domain, allowing the calculation of metrics such as maximum frequency, mean frequency, variance, standard deviation, signal power, skewness, and kurtosis. This is achieved through a custom Python script that reads CSV files from the rotary machine fault simulator, applies FFT to each sensor's data, computes the statistical metrics, and saves the extracted features into new CSV files. In the Time Domain feature extraction, the script calculates statistical metrics directly from the raw sensor data, including mean, variance, standard deviation, root mean square (RMS), peak-to-peak values, skewness, and kurtosis. Both feature extraction methods are designed to systematically process all input files, outputting structured datasets ready for machine learning models.

The machine learning component is implemented using the scikit-learn library, incorporating a variety of classifiers such as Decision Tree, Support Vector Machine (SVM), Logistic Regression, Linear Regression, and Neural Networks. These models are utilized for both fault detection (binary classification of fault vs. no-fault) and fault categorization (multiclass classification of specific fault types). The machine learning pipeline involves splitting the data into training and testing sets, standardizing the data (where applicable), and training each model on both Frequency Domain and Time Domain features. Performance metrics, including accuracy, precision, recall, F1-score, confusion matrices, mean squared error, and R2 score, are automatically generated for each model and displayed to the user.

A key enhancement to the system is the **interactive terminal-based menu**, which significantly improves usability and efficiency. This menu allows users to navigate the entire process through simple numbered prompts. First, users select the dataset they wish to analyze (25, 50, or 75 RPM), then choose the type of feature extraction (Frequency Domain or Time Domain), and finally select a machine learning model from a list of five options. Once selections are made, the system automatically runs the corresponding processes and displays detailed results on-screen, including classification metrics and confusion matrices. The menu loop also enables users to quickly iterate through multiple configurations without rerunning or modifying any code manually. This interactive design makes the tool accessible for users of all experience levels and supports extensive experimentation with minimal effort.

The system interfaces seamlessly between hardware and software. Data collected from the hardware is formatted for software processing, and the machine learning models interact directly with the feature extraction output. The modular design of the software allows for easy updates and maintenance, with distinct modules handling data acquisition, preprocessing, model training, and evaluation.

5.4 Detailed Architecture

Our program starts by importing all of the machine learning libraries, including Pandas, Scikit Learn, and NumPy, that are used throughout our project to easily manipulate databases, create machine learning models, and create performance metrics. Pandas is a Python Library that is used to manipulate and analyze data. We used this library for feature extraction along with NumPy which is used for mathematical functions and manipulating multidimensional arrays. Scikit Learn is a machine learning and statistical modeling library we used to create our machine learning models. Additionally, it has multiple performance metric functions that measure the accuracy, precision, recall, F1-score, and more for multiple machine learning models. These libraries have been installed and are used with our feature extraction and model development programs.

Next, we downloaded the datasets into our program to perform feature extraction. Feature extraction is the process of transforming raw data into a set of useful features that can be used by machine learning models to classify data. The data we are performing feature extraction on consists of three different folders composed of multiple CSV files. Each folder holds multiple CSV files of Machine Fault Simulator trial data. The folders are separated by RPM speed including 25, 50, and 75 RPM. Each folder has 950 CSV files consisting of different machine fault trials. To download each of these files from the folder, we used the folders path and the glob function and put each file into a DataFrame. We then added each of these DataFrames to an array for each folder. Each DataFrame then went through the feature extraction process and was simplified to a single row consisting of the feature-extracted data and placed into a final DataFrame that was converted into a CSV file used for training and testing our machine learning models.

Next, we began working on the feature-extraction. The first thing we noticed about our CSV files was, there were multiple repeated rows of time data. We only needed one column of time data, since it was the same for every column. So, we deleted eight of the repeated columns from each DataFrame, halving the number of rows in our DataFrame. After deleting any unnecessary columns, we created our feature extraction method. Before creating our feature extraction method, we first created an array of all the column names that would be created from our method. Then we created a DataFrame called "df_extracted" and set this to null with the column names set to the column names array we created earlier. We have two separate programs for feature extraction: a Time-Domain feature extraction program and a Frequency-Domain feature extraction program. The previous steps for each method are the same, but the data manipulation is different for each program. For the Frequency-Domain feature extraction, we took in the DataFrame of the data we are manipulating, the DataFrame that will store the feature extracted data, the category of fault as an integer, and whether there was a fault detected or not as an integer as parameters. We went through each column in the DataFrame, transformed it into a Fast Fourier Transform, and found the magnitude of the data. We then found the maximum, mean, variance, standard deviation, signal power, skew, and kurtosis of each magnitude value. We added these values into an array called df_freq_vals which holds all the feature-extracted data for a single CSV file. We then appended the fault_detected and fault_category values to the array and appended the array to the DataFrame of feature-extracted data. We repeated this process for every DataFrame in the array and then returned the feature-extracted DataFrame. The Time-Domain feature extraction function is similar to the Frequency-Domain function. They take in the same values and each goes through each column in the DataFrame, but rather than using the Fast Fourier Transform it uses the values of the columns, dropping any null values and converting them to numeric values, and finds the max,

mean, variance, standard deviation, kurtosis, skew, precision time protocol and root square mean of each column. These feature-extracted values are appended to the `df_time_vals` array along with the `fault_detection` and `fault_category` values and then added to the DataFrame of feature-extracted values. To get the category and detection values we had to manually add these in since they were not added to the CSV files. The detection values were equal to 1 in all cases, meaning there was a fault detected, except for the no-fault trial CSV files. For the category value, there were 25 trials per fault, so the category was set to 1 and then incremented after every 25th CSV file was put through the feature extraction method. These DataFrames were then converted into CSV files that can be used for our machine learning models.

Once our data had been transformed, we worked on programming, training, and testing our machine-learning models. The machine learning models we have chosen to program are the SVM, Decision Tree, Linear Regression, Logistic Regression, and Neural Network. SVM and decision trees have already been studied for this project, but we are looking at them again with the frequency-domain features to see if there is any change in the model's accuracy. Additionally, we are studying three new models: Linear Regression, Logistic Regression, and Neural Networks to see if there is any improvement in the accuracy of fault prediction using these models compared to the previous models. The SVM model classifies data by finding the hyperplane that maximizes the distance between each class in an N-dimensional space [9]. We are using the SVM models to determine between the fault classes and non-fault classes, so we are finding the hyperplane that separates these classes using our data. The Decision Tree model uses data to create a tree-like structure that uses statistics to split data based on features used for classification [10]. We are using the data from our DataFrames to find the attributes that best determine how to classify our data and use them to build our tree. Linear regression determines the connection between dependent and independent variables of a dataset, and logistic regression predicts the outcome of a data point based on independent variables by applying logistic functions [10]. These two functions are looking at the independent and dependent functions of the datasets and then finding a linear and logistic function that can be used to predict the class of the data point. Neural Networks use a series of algorithms to recognize patterns and relationships in data [11]. There is an input layer, output layer, and hidden layers used to process the data [11]. The Neural Network model takes in the datasets as input and uses them to adjust the weights and biases of the hidden layers to output the correct classification for the data points. To create these models, we are using the Pandas and Scikit-learn libraries in Python. These libraries have functions that can perform data train and test split, create models, fit models, and train and test models. For each of these machine learning model types, we are training twelve different models with different test data. Three groups are trained on different RPM speeds, then of the four models in those groups, we are training a combination of detections vs. categorization classification and frequency-domain vs time-domain features.

To determine the adequacy of each machine learning model we are measuring their performance. These measurements include accuracy, precision, recall, F1-score, and the confusion matrix printed for the decision tree, SVM, logistic regression and neural network models. The accuracy represents the number of correctly classified data points divided by the total number of data points. This is a great measurement for determining the credibility of datasets whose classification classes are equally distributed, but if they are not distributed equally then the accuracy measurement can give an inaccurate performance measure for the model. So, we are using precision, recall, and F1-score to gather more information about the model's performance. Precision is the proportion of a

model's positive classifications that are actually positive, this is given by the equation $\text{true positive} / (\text{true positive} + \text{false positive})$ [12]. Recall is the proportion of all positive classifications that are correctly classified as positive, given by the equation $\text{true positives} / (\text{true positives} + \text{false negatives})$ [12]. The F1 score is the mean of the recall and precision values, given by the equation $2 * (\text{Precision}) * (\text{Recall}) / (\text{Precision} + \text{Recall})$. The confusion matrix shows the predictions of all classes of the datasets in a matrix form. This gives us specific insights into the classification of each data point including their true class and predicted class. This helps us make conclusions about how the machine learning model predicted certain classes and if improvements can be made based on this new information. Each of these performance metrics is printed out for each of the machine learning models so they can be used to compare results and see which models are best for detecting specific faults. There are a total of 60 different performance metric results (one for each model). These performance metrics are printed into a typescript that can be used for future research and interpretation. To determine the performance metrics, we will use the Scikit-learn libraries to use functions that will determine the accuracy, precision, recall, f1 score, and confusion matrix.

The final task of this project was to create a menu for our machine-learning model code to make the program user-friendly and easier to read. For the feature extraction programs, the user can choose between the three datasets, 25, 50, and 75 RPM, by selecting "1", "2" or "3" from the menu. If the user chooses "1" they will get the feature-extracted information for the 25 RPM dataset. If the user chooses "2" they will get the feature-extracted information for the 50 RPM dataset. If the user chooses "3" they will get the feature-extracted information for the 75 RPM dataset. The user will then be prompted to choose how many files they would like to test with. After the user has input this information, the program will run and print a CSV file with the feature-extracted data. For the machine learning model program, the user must first choose which feature extraction data they would like to use. The menu prompts the user to choose either "1" for the 25 RPM data, "2" for the 50 RPM data, or "3" for the 75 RPM data. Then it prompts the user to choose between feature extraction methods, "1" for frequency-domain and "2" for time-domain feature extraction. Finally, it prompts the user to choose which model they would like to use and get the performance metrics for. The user can choose "1" for Linear Regression, "2" for Logistic Regression, "3" for Decision Tree, "4" for SVM, and "5" for Neural Network. The program will then print out the results that match the user's input and then ask if they would like to continue and choose another model to look at. If they type "1" the program will repeat the previous actions, if they type "0" it will end the program. For all of these inputs, if the user inputs information not from the menu such as "5" or "24" it will prompt the user again until the user input matches the values from the menu. This allows the user to easily understand the program and gather specific data according to their specific needs without going through unnecessary information.

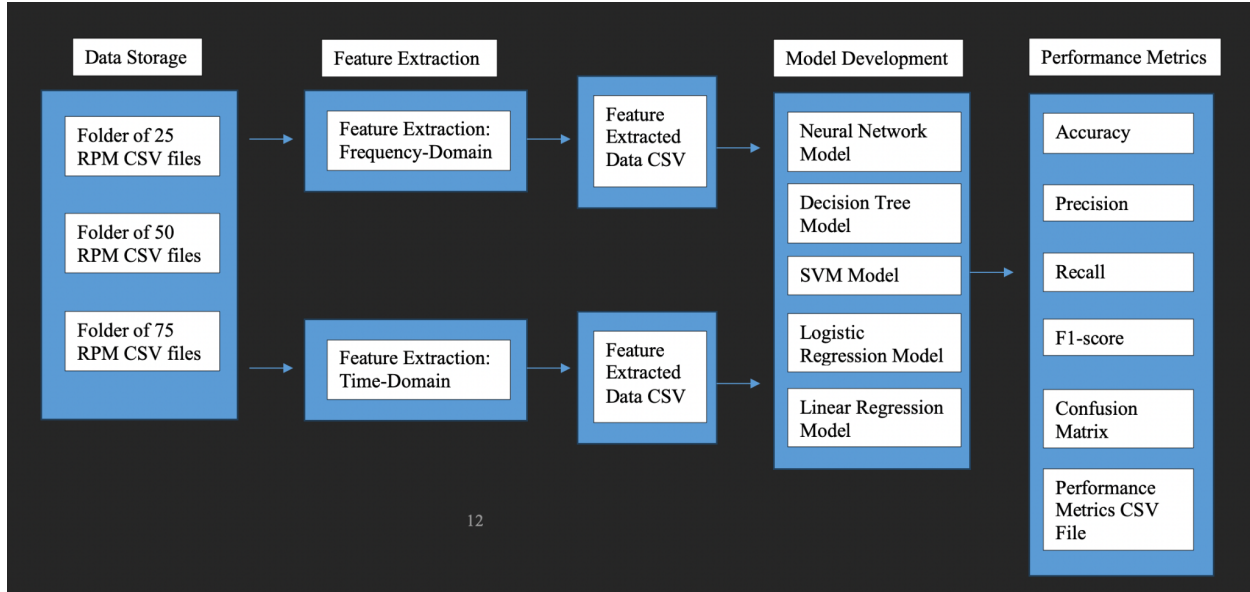


Figure 1. Diagram of High Level Architecture

5.5 Data Management and Storage

Our program is designed to efficiently handle, process, and store sensor data collected from the rotary fault simulator. The goal is to ensure a structured approach to fault detection while making data accessible for machine learning analysis. The data acquisition process collects real-time accelerometer readings and rotational speed measurements, which are continuously recorded to track machine conditions. These readings are stored in CSV files to maintain compatibility with the machine-learning models used in the project. Each entry is timestamped and converted from an analog signal into a digital format to ensure accuracy and consistency. This structured approach allows for smooth integration between data collection, preprocessing, feature extraction, and classification.

The collected data is stored in separate CSV files, each serving a different purpose: raw sensor data, extracted features, and classification results. The raw sensor data contains vibration readings recorded at different rotational speeds (25, 50, and 75 RPM) under both normal and faulty conditions. To prepare the data for machine learning, preprocessing is applied, which includes handling missing values, filtering out noise, and normalizing the readings to ensure consistency. After preprocessing, statistical features such as mean, variance, standard deviation, skewness, and kurtosis are extracted and saved in separate datasets. These extracted features help identify patterns in machine behavior, improving the accuracy of fault classification. Once the machine learning models have been trained and tested, their classification results are also stored, allowing for performance comparisons and further refinement.

The system follows a structured data processing workflow based on the project code. First, it imports raw sensor readings from CSV files containing both frequency-domain and time-domain data. Preprocessing techniques clean the data and prepare it for feature extraction. Time-domain

analysis focuses on direct statistical values from the vibration signals, while frequency-domain analysis applies the Fast Fourier Transform (FFT) to detect patterns that may indicate faults. These extracted features are then used as input for the machine learning models, improving their ability to distinguish between normal and faulty machine conditions.

Since the project involves large amounts of sensor data, the system processes it in smaller batches rather than loading everything into memory at once. This prevents excessive memory consumption and ensures smooth operation, even when working with high data volumes. Additionally, the system stores extracted feature sets separately from raw data to prevent redundant computations, reducing processing time and improving runtime efficiency. This setup is particularly useful for real-time fault detection, as it keeps the system responsive while handling large amounts of incoming data.

For long-term usability, the system includes options to export processed datasets, classification results, and performance evaluation metrics. Users can generate and save classification reports, confusion matrices, and statistical summaries for further analysis. By organizing data in structured storage formats, previous results can be revisited, allowing researchers and maintenance teams to track machine performance trends over time. The ability to archive datasets also enables the comparison of historical data with new sensor readings, supporting continuous improvements to the machine learning models.

Maintaining data security and integrity is an important part of the storage process. Each data file is labeled with metadata and timestamps, ensuring proper organization and traceability. The system also organizes data into structured folders based on different RPM conditions, making it easier to retrieve specific datasets when needed. Additionally, version control mechanisms track updates and modifications to ensure that the datasets remain consistent and prevent data loss. Periodic backups can also be implemented to further protect stored data.

Overall, the data management and storage system is designed to provide a structured, efficient, and scalable workflow that integrates seamlessly with the machine learning models. By organizing data efficiently, processing it in batches, and ensuring secure storage, the system enables reliable real-time fault detection while keeping the data accessible for researchers, students, and maintenance teams working with rotary machine diagnostics.

6.0 Characterization of Vibration Fault

6.1 Failure Vibration Modes In Rotating Machinery

Rotating machines such as motors, turbines, and industrial drives are plagued by many failure modes including bearing defects, gear tooth failure, shaft misalignment, and imbalance. All these failure modes impart a distinct vibration signature on the machine, which forms the basis of vibration-based fault detection. In fact, vibration analysis can detect around 90% of faults in rotating equipment because each component (bearing, gear, shaft, etc.) produces specific patterns of vibration when faulty. Common faults that can be detected by vibration monitoring are:

- Rolling-element bearing defects** – e.g. inner race spalling, outer race spalling, rolling element defect (ball/roller), or cage damage. These create specific frequencies known as bearing characteristic frequencies: outer race defects at BPFO (Ball Pass Frequency Outer), inner race defects at BPFI (Ball Pass Frequency Inner), rolling element defects at BSF (Ball Spin Frequency), and cage instability at FTF (Fundamental Train Frequency). A spalled bearing will produce cyclical impact pulses as each roller passes over the defect, inducing high-frequency resonances. The vibration spectrum of a bearing fault typically has peaks at the BPFO/BPFI and harmonics, sidebands at the shaft rotation frequency if the fault is modulated by the shaft motion. Incipient bearing faults generate very low-energy, high-frequency signals, and therefore special techniques like envelope analysis or high-frequency demodulation are needed to detect them. As the bearing damage progresses, these frequency peaks become larger and other harmonics become apparent, indicating further fault development.
- Gearbox faults** – Gear defects include chipped or broken teeth, tooth wear (pitting), eccentric gears, and gear misalignment. A defective gear locally causes a disturbance in the otherwise periodic meshing vibration. The gear mesh frequency (GMF, typically a high-frequency related to teeth engagement rate) will have sideband frequencies at a spacing of the shaft rotation rate if faults are present. For example, a curved shaft or eccentric gear produces modulation of the GMF and gives rise to sidebands at $1\times$ shaft RPM around the mesh frequency peaks. A fractured or cracked tooth gives rise to a per-revolution excitation that might drive natural frequencies of gears and yield new spectral lines at gear or pinion resonances. In general, gear defects manifest as elevated vibration at the GMF harmonics and presence of sidebands or resonant peaks. The amplitudes of the GMF and the first few of its harmonics will greatly amplify due to tooth fracture or heavy wear, and can introduce sidebands at fault rotation frequency. Table 1 shows some of the typical vibration signatures for a range of different fault types.

Fault Type	Characteristic Vibration Signatures
Bearing inner race	Harmonic series at BPFI (inner race pass frequency); may have 8–10 harmonics, often with $1\times$ RPM sidebands. Envelope spectrum reveals BPFI peaks clearly.
Bearing outer race	Harmonic series at BPFO (outer race pass frequency); 8–10 harmonics of BPFO with relatively constant spacing. Sidebands less pronounced than inner race.

Fault Type	Characteristic Vibration Signatures
Bearing roller/ball	Harmonics of BSF (ball spin frequency), often with the largest amplitude at a particular harmonic indicating defect location. High-frequency resonances excited by impacts.
Gear tooth wear	Raised baseline at gear mesh frequency (GMF) and its harmonics. Possible excitation of gear natural frequencies due to friction. Multiple sidebands around GMF if wear is uneven.
Broken gear tooth	Pronounced sidebands around GMF at $1\times$ gear rotation frequency (once-per-rev impact). May see a drop in one specific tooth mesh harmonic and noise at resonance frequencies.
Shaft misalignment	High $2\times$ running speed vibration (twice per rotation) and elevated $1\times$ as well. Can also excite gear mesh harmonics (e.g., $2\times$ GMF) due to cyclic load variation as misaligned shafts engage.
Imbalance (eccentric rotor)	Dominant $1\times$ running speed vibration (high amplitude at fundamental frequency). Little to no higher harmonics unless coupled with misalignment.

Table 1: Common rotating machinery faults and their vibration signature characteristics.

Understanding these signatures is crucial for fault diagnosis. By identifying frequency-domain patterns corresponding to known fault frequencies, one can determine not only that a fault exists but often infer its type (e.g. inner vs outer race bearing fault). However, capturing these signatures reliably can be challenging if the signal is noisy or the machine operates under varying speed/load conditions.

6.2 Fault Detection Techniques And Algorithms

Signal Processing for Feature Extraction: Traditional fault detection relies on signal processing to enhance fault-indicative patterns in vibration data. This typically involves transforming the raw time-domain vibration signal into more informative representations. Common techniques include:

- **Frequency analysis (FFT)** – Computing spectra to find peaks at characteristic frequencies (bearing fault frequencies, gear mesh frequencies, etc.). This is effective for stationary conditions but can miss time-varying behavior or very subtle faults masked by noise. For example, low-speed machines produce fault frequencies in the very low Hz range, where background vibration can obscure the peaks. Conversely, incipient bearing faults generate ultrasonic-frequency bursts that may not noticeably raise the low-frequency FFT

components. Thus, while FFT-based features (peak amplitudes, frequency ratios) are widely used, they often need preprocessing steps to be effective.

- **Envelope analysis (amplitude demodulation)** – A powerful method especially for rolling element bearings. By applying a high-pass filter and extracting the envelope of the vibration (often via Hilbert transform), one can isolate repetitive high-frequency impact signatures. The envelope spectrum then clearly shows the bearing fault frequencies even if the raw spectrum was dominated by other components. Envelope analysis is considered key for early detection of bearing faults, as it boosts the weak repetitive impulses before they grow large. Many modern approaches use *adaptive envelope extraction* or spectral kurtosis to choose the optimal band for demodulation, maximizing the signal-to-noise ratio of fault impulses.
- **Time-domain statistical features:** Simple metrics calculated from the time signal (peak, RMS, kurtosis, crest factor, etc.) can indicate fault presence. For instance, bearing faults typically cause an increase in vibration kurtosis (due to spiky impacts). These features are quick to compute and often used in machine learning models. In fact, time-domain features often complement frequency features; a comprehensive feature set might include RMS (overall energy), kurtosis (impulsiveness), and specific spectral amplitudes. Recent studies leverage both domains: *e.g.* Jiao *et al.* (2023) combine time- and frequency-domain information using wavelet transforms and report improved fault classification on imbalanced data.
- **Time-frequency analysis:** When machine speed or load varies, fault signatures can shift in frequency. Techniques like Short-Time Fourier Transform (STFT), Wavelet Transform, or Wigner-Ville distribution are used to capture transient or speed-varying phenomena. For example, an *order analysis* (frequency normalized by RPM) can track gear or bearing fault frequencies under changing speed. One study introduced an iterative SVD-based in-band noise reduction combined with envelope order analysis to extract fault features under variable speeds, highlighting the trend towards hybrid techniques that address noise and non-stationarity simultaneously.

Noise Mitigation: A critical pre-processing step in fault detection is noise reduction. Industrial environments introduce substantial noise (background vibrations, sensor noise, etc.) that can mask fault indicators. Common noise mitigation approaches include:

- **Filtering:** Using band-pass filters to isolate frequency bands where fault content is expected (for instance, filtering around bearing BPFO/BPFI ranges, or around gear mesh frequency). High-frequency noise can be reduced with low-pass filters; likewise, very low-frequency drift can be removed with high-pass filters. Adaptive filters and spectral subtraction methods can target specific interference frequencies. The Savitzky–Golay

filter, a polynomial smoothing filter, is also used to denoise time-domain signals while preserving the shape of pulses.

- **Wavelet denoising:** Wavelet transforms can separate a signal into multi-scale components. By thresholding small wavelet coefficients (assumed to be noise) and reconstructing the signal, one achieves a de-noised signal that retains sharp transients from faults. Wavelet-based denoising has proven effective for enhancing weak fault pulses in bearings.
- **Ensemble averaging and synchronous averaging:** For gearboxes, averaging vibration signals over many rotations can cancel out random noise and reinforce periodic content (like gear mesh). Synchronous time averaging aligned to shaft rotation is classic for isolating gear tooth fault effects from noise and from other gears.

In practice, a combination of these signal processing techniques is used to extract a rich set of features.

Fault Classification Algorithms: Once features are extracted, classification algorithms are employed to automatically identify the fault condition (or predict remaining life, as discussed later). Over the years, a wide range of fault classification methods have been explored:

- **Machine Learning:** Traditional classifiers like Support Vector Machines (SVM), decision trees, random forests, K-Nearest Neighbors (KNN), and neural networks (MLP) have all been applied to vibration-based fault diagnosis. For example, Huynh & Min (2024) extracted 58 time and frequency features from sensor data and achieved 99.8% fault classification accuracy using an SVM classifier. Feature selection reduced the input to 11 key features with minimal accuracy loss. In another study, an AdaBoost ensemble of decision trees outperformed SVM and KNN in classifying bearing faults, highlighting that ensemble methods can be very effective. These algorithms usually require careful tuning and may struggle if the feature space is not discriminative; this has driven research into better feature extraction as noted above.
- **Deep Learning:** In recent years, deep learning approaches have gained prominence for fault detection. Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNN/LSTM) can learn feature representations directly from raw signals or spectrograms, reducing the need for manual feature engineering. Researchers have reported excellent results using CNNs to automatically extract multi-scale features; for instance, a multi-branch CNN achieved 99.9% accuracy in classifying bearing faults under a constant speed/load condition. Even under varying loads, the CNN was more robust than conventional methods, achieving a few percentage points higher accuracy than comparable shallow learners. Deep learning models (1D CNNs, autoencoders, etc.) can incorporate

time-frequency input (like wavelet scalograms or STFT spectrograms) to capture complex patterns. However, deep models require large labeled datasets and are often treated as "black boxes," so combining them with signal processing domain knowledge (as in hybrid models) is an active area of research.

- **Prognostic Algorithms:** While many classification algorithms focus on identifying the *current* fault state (diagnosis), some are used for *prognosis* – predicting how the fault will evolve. Classification can be extended to predict discrete levels of fault severity (e.g., health stage 0,1,2,...). For example, an SVM might categorize bearings into health index levels (healthy, slight damage, moderate, severe) based on features. This can be seen as an intermediate approach between pure diagnosis and continuous RUL prediction.

Predictive Health Monitoring and Bearing Life Prediction: Beyond fault detection, the literature on Predictive Health Monitoring (PHM) deals with forecasting the remaining useful life (RUL) of components. This is especially critical for bearings, as predicting bearing life allows scheduled replacements before catastrophic failure. Approaches to RUL prediction generally fall into a few categories:

- **Physics-based models:** Using analytical models of degradation (e.g., Paris' law for crack growth, or empirical bearing life formulas) to estimate life. For bearings, the classic L10 life calculation (based on load and fatigue life models) provides a baseline, but it doesn't account for real-time condition monitoring data. More advanced physics-based methods might simulate how spall size grows per hour given stress conditions, etc. These methods require detailed knowledge of material properties and failure mechanisms.
- **Statistical models:** Methods like proportional hazard models or state-space models (Kalman filters, particle filters) that treat wear as a stochastic process. These may use vibration feature trends as inputs to update a probability distribution of remaining life. For example, a hidden Markov model could represent healthy vs degraded states and estimate time to absorption (failure).
- **Data-driven machine learning:** Using regression or survival analysis on historical run-to-failure data. Early works applied algorithms like support vector regression or neural networks to map condition indicators (vibration levels, temperature, etc.) to time-to-failure. These had limitations, as noted by Guo *et al.* (2017): traditional ML needed manual feature engineering and lacked generalization to new operating conditions. Today, deep learning dominates RUL research for bearings. Recurrent networks (LSTM/GRU) and CNNs can learn from sequences of sensor data to predict how much life is left. For instance, Sun *et al.* proposed a CNN-Transformer hybrid for bearing RUL that outperformed conventional models on benchmark datasets. Generally, deep models automatically capture the nonlinear progression of damage, which can improve prediction accuracy if sufficient training data is available.

- **Hybrid methods:** Combining physical insights with data-driven models, such as using model-based features in a neural network, or updating neural predictions with Bayesian filters. These aim to incorporate domain knowledge (like known failure progression patterns) to improve extrapolation of RUL beyond observed data.

In summary, the state-of-the-art in rotating machine fault monitoring combines advanced signal processing (to handle noise and non-stationarity) with sophisticated classification and regression algorithms (to diagnose faults and predict life). Classic vibration analysis methods (spectral peaks, envelope detection, etc.) remain highly relevant and are often the foundation, but they are increasingly augmented by machine learning and AI for automated detection and prognostics.

6.3 Metrics and Evaluation for Fault Classification

In the current project, multi-class fault classification performance was primarily judged by overall accuracy. Accuracy (the fraction of correct predictions) is a useful metric, but it has limitations, especially for multi-class and imbalanced fault detection problems:

- **Class Imbalance Issues:** If some fault types occur much more frequently than others, accuracy can be misleading. A classifier could predict only the majority class and still achieve high accuracy, yet completely fail to detect the rare faults. For example, in our 50 RPM dataset, there were 188 instances of one class and only 7 of another; a model that always predicts the majority class would be 96% accurate but obviously useless for detecting the minority fault. This scenario actually occurred – one model achieved 96% accuracy while scoring 0% recall on the minority class. Overall accuracy did not reflect that the model missed *all* instances of the rare fault.
- **Multi-Class Confusion:** Accuracy collapses all classes into a single number, obscuring which faults are misclassified. In a multi-class fault diagnosis (e.g., distinguishing 10 different fault types), it is important to know per-class performance. A confusion matrix and per-class metrics are needed to identify if, say, "bearing inner race and outer race faults are often confused," or if a particular fault type is never recognized.

To address these issues, alternative evaluation metrics should be used in addition to accuracy:

- **Macro-Averaged F1 Score:** The F1 score is the harmonic mean of precision and recall. By computing it per class and then taking an unweighted average (macro-F1), we treat each class equally. Macro-F1 is sensitive to the performance on minority classes. If any class has poor precision/recall, the macro-F1 will drop. In our example above, the model with 96% accuracy had a macro-F1 around 0.49 because it scored poorly on one class – this revealed the imbalance issue clearly. A high macro-F1 (close to 1.0) indicates the classifier is performing well across *all* fault categories. We will incorporate macro-F1 as a key metric for multi-class models.

- **Balanced Accuracy (Class-Weighted Accuracy):** This metric is defined as the average of the recall (sensitivity) for each class. It effectively gives each class equal weight, counteracting imbalance. Balanced accuracy is equivalent to accuracy if the test set is balanced, but on imbalanced data it provides a more fair assessment. A low balanced accuracy in contrast to a high raw accuracy is a red flag. (Note: sometimes "class-weighted accuracy" refers to weighting by class importance; here we specifically mean balanced accuracy, weighting each class equally).
- **Precision and Recall per Class:** Instead of a single number, providing the precision (how many predicted faults were correct) and recall (how many actual faults were caught) for each class is very informative. For critical faults (say a catastrophic failure mode), one might prioritize recall (catch all occurrences) even at expense of precision.
- **Confusion Matrix:** This is a tabular breakdown of predicted vs actual class counts. It allows pinpointing which faults are getting confused. For instance, a confusion matrix might show that all instances of "gear misalignment" were misclassified as "unbalance," suggesting those signatures were too similar in the feature space. By examining the matrix, we can target specific classification errors for improvement (e.g., adding features to distinguish those two faults).
- **Top-K Accuracy:** In scenarios where the classifier outputs a ranked list of fault hypotheses (common in deep learning models), top- k accuracy is useful. Top-2 accuracy, for example, checks if the correct fault type is either the first or second guess of the model. This can be meaningful in a diagnostic aid context: if the true fault is usually among the model's top 2 or 3 predictions, the tool may still be useful for maintenance decisions. Top-K accuracy becomes relevant when many fault classes exist or when the model is used to suggest possible causes.

In summary, we propose to track macro-F1, balanced accuracy, and present the confusion matrix for our fault classifier, in addition to the standard accuracy. These metrics will give a more nuanced view of performance. An ideal model will have high values for all (indicating it performs well across all fault types). If accuracy is high but macro-F1 is low, we know the model is skewed toward certain classes. Likewise, presenting precision/recall for each class will highlight any fault types that need more attention (data or features).

6.4 Preprocessing And Feature Extraction Results

Raw vibration data (sensor readings over time) were first cleaned by removing initial transient intervals. The data were then segmented (one segment per test cycle) and labeled with `fault_detected` (0=no-fault, 1=fault) and `fault_category` (integer code for fault type). Two types of features were extracted:

- **Time-Domain Features:** For each of nine sensors (Tachometer, Motor, three-axis accelerometers on bearings B1 and B2, and a Gearbox sensor), the following statistics were computed on the time-series: max, mean, variance, standard deviation, root-mean-square (RMS), kurtosis, skewness, and peak-to-peak (max–min). These features characterize the waveform shape and amplitude distribution.
- **Frequency-Domain Features:** A fast Fourier transform (FFT) was applied to each sensor’s time signal to obtain spectral magnitudes. From these spectra, for each sensor we computed: max magnitude, mean magnitude, variance, standard deviation, mean-squared magnitude (signal power), kurtosis, and skewness of the spectrum. These features capture frequency content such as dominant harmonics or noise.

All extracted features, along with the labels, were compiled into feature tables for each RPM and domain type. This preprocessing pipeline follows common practice in vibration analysis.

6.5 Machine Learning Models

We evaluated five models on two tasks: (1) **Fault Detection** (binary classification of fault vs. no-fault) and (2) **Fault Categorization** (predicting the fault category among multiple classes). The models were:

- **Decision Tree**
- **Support Vector Machine (SVM)**
- **Logistic Regression**
- **Linear Regression (Ridge Regression)**
- **Neural Network**

Models were trained and tested on each RPM’s dataset separately. Classification metrics (accuracy, precision, recall, F1-score, confusion matrix) were computed for detection and categorization. For regression, we report MSE and R^2 . No class rebalancing was applied.

6.5.1 Fault Detection Performance

Model	25 RPM Time (%)	25 RPM Freq (%)	50 RPM Time (%)	50 RPM Freq (%)	75 RPM Time (%)	75 RPM Freq (%)
Decision Tree	100	95.4	100	100	100	100
SVM	96.4	96.4	96.4	96.4	96.4	96.4
Logistic Regression	96.4	96.4	96.4	96.4	96.4	96.4
Linear Regression	100 ($R^2=1.00$)	95.3 ($R^2\approx-0.05$)	100 ($R^2=1.00$)	100 ($R^2=1.00$)	100 ($R^2=1.00$)	100 ($R^2=1.00$)

Model	25 RPM Time (%)	25 RPM Freq (%)	50 RPM Time (%)	50 RPM Freq (%)	75 RPM Time (%)	75 RPM Freq (%)
Neural Network	100	96.4	100	100	100	100

Confusion matrices confirm that Logistic Regression and SVM missed all “no-fault” samples, classifying all as “fault”, giving high accuracy but poor recall for minority class.

6.5.2 Fault Categorization Performance

Model / Feature	F1-Score (categorization)
Decision Tree (time)	0.98
Linear Regression (time)	N/A ($R^2=1.00$)
SVM (time)	≈ 0.07
Logistic (time)	≈ 0.05
Neural Net (time)	0.72
Decision Tree (freq)	0.57
SVM (freq)	~ 0.00
Logistic (freq)	~ 0.05

Categorization was more difficult due to the number of classes ($\sim 30+$) and class imbalance.

Time-domain features consistently provided better results than frequency-domain features for detection. Decision Trees and Neural Networks were the most effective models. Logistic Regression and SVM underperformed due to class imbalance. The models overfit to the dominant class when no countermeasures (e.g., SMOTE) were used. In future work, combining time and frequency-domain features may improve robustness. Denoising and more sophisticated time–frequency techniques (like wavelets) may be helpful for early-stage fault identification.

Time-domain statistical features, combined with Decision Trees or Neural Networks, yielded the best performance in fault detection. Frequency-domain features were slightly less effective. Fault categorization remains an open challenge due to extreme class imbalance and a high number of fault classes. Improving data collection, rebalancing, and exploring new features and models are recommended directions for future research.

7.0 Development Plan

7.1 Tasks

1. Research about different types of feature extraction methods and machine learning model development
2. Research information about the features in the CSV files to find the best way to utilize them
3. Download python, anaconda environment, and machine learning libraries (scikit-learn, Pandas, NumPy, ect.)
4. Create Github to share project code with our group
5. Download CSV files of all 38 faults into program
6. Begin feature extraction for time-domain features for the 25 RPM Trials CSV files
7. Begin feature extraction for time-domain features for the 50 RPM Trials CSV files
8. Begin feature extraction for time-domain features for the 75 RPM Trials CSV files
9. Begin feature extraction for frequency-domain features for the 25 RPM Trials CSV files
10. Begin feature extraction for frequency-domain features for the 50 RPM Trials CSV files
11. Begin feature extraction for frequency-domain features for the 75 RPM Trials CSV files
12. Create CSV files with newly time-domain extracted features for Machine Model testing
13. Create CSV files with newly frequency-domain extracted features for Machine Model testing
14. Begin programming SVM model
15. Begin programming Linear Regression Model
16. Begin programming Logistic Regression Model
17. Begin programming Neural Network model
18. Begin programming Decision Tree model
19. Print performance metrics and confusion matrix for SVM model
20. Print performance metrics and confusion matrix for Linear Regression Model
21. Print performance metrics and confusion matrix for Logistic Regression Model
22. Print performance metrics and confusion matrix for Neural Network Model
23. Print performance metrics and confusion matrix for Decision Tree Model
24. Create menu for easy use of code for researchers
25. Statistical analysis of results

7.2 Schedule

Task Description	Assigned Members	Est Start	Est End	Actual Start	Actual End
Background information and research into feature extraction methods	Victor	1/01	1/17	01/1	01/17
Downloading Python, Anaconda, and libraries	Victor	1/01	1/17	01/17	01/17

Create Github	Victor	1/01	1/17	01/15	01/15
Downloading all CSV files	Victor	01/20	01/22	01/24	01/28
Begin feature extraction for time-domain features for the 25 RPM Trials CSV files	Victor	01/13	02/06	01/29	02/26
Begin feature extraction for time-domain features for the 50 RPM Trials CSV files	Victor	01/13	02/06	01/29	02/26
Begin feature extraction for time-domain features for the 75 RPM Trials CSV files	Victor	01/13	02/06	01/29	02/26
Begin feature extraction for frequency-domain features for the 25 RPM Trials CSV files	Victor	01/13	02/06	01/29	02/07
Begin feature extraction for frequency-domain features for the 50 RPM Trials CSV files	Victor	01/13	02/06	01/29	02/07
Begin feature extraction for frequency-domain features for the 75 RPM Trials CSV files	Victor	01/13	02/06	01/29	02/07
Create CSV files with newly time-domain extracted features for Machine Model testing	Victor	01/13	02/06	01/29	02/26
Create CSV files with new frequency-domain extracted features for Machine Model testing	Victor	01/13	02/06	01/29	02/10
Begin programming SVM model	Victor	02/6	02/19	02/26	03/03
Begin programming Linear Regression Model	Victor	02/6	02/19	02/27	03/04
Begin programming Logistic Regression Model	Victor	02/6	02/19	02/27	03/04
Begin programming Neural Network model	Victor	02/6	02/19	02/27	03/18
Begin programming Decision Tree model	Victor	02/6	02/19	02/10	02/20
Print performance metrics and confusion matrix for SVM model	Victor	02/20	02/26	02/26	03/03
Print performance metrics and for Linear Regression Model	Victor	02/20	02/26	02/27	03/04
Print performance metrics and confusion matrix for Logistic Regression Model	Victor	02/20	02/26	02/27	03/04

Print performance metrics and confusion matrix for Neural Network Model	Victor	02/20	02/26	02/27	03/18
Print performance metrics and confusion matrix for Decision Tree Model	Victor	02/20	02/26	02/22	02/26
Create Menu for easy use of code for Researchers.	Victor	02/27	03/19	03/17	04/07
Review Performance Metrics	Victor	02/27	3/19	02/26	04/14
Statistical analysis of results	Victor	03/20	03/26	03/18	04/14

7.3 Deliverables

- CSV files of datasets used to train and test the models: Our finished project will use CSV files with data to train our machine learning models. It is important to keep these files stored for future use and add any additional trials we may test in the future.
- Python code for feature extraction, machine learning models, and performance metrics: This is the code for our machine learning models that will be used for future research and projects. This code will investigate five different machine learning models and provide useful information about how machine learning models can be used to detect faults in rotary machines. This software can be used for current data and future data and will be a useful tool for learning more and developing more machine learning programs for fault detection.
- Raw performance metrics output: This will return the performance metrics for each machine learning model. This will be useful for researchers to look at and see where specific models can improve or which models are best for detecting certain faults. They can analyze the systems and make decisions on the future development of these models.
- Statistical analysis paper: This paper will be a brief analysis of our machine learning models and their results. This will demonstrate the usefulness of our models in discovering information about how machine learning models can be useful in machine fault detection. We will look at multiple different values and make conclusions about which models performed the best and how as well as which feature extraction techniques are the most useful.

8.0 Key Personnel

Victor Manuel Cabaleiro Valado – Victor is a senior of both Computer Science and Electrical Engineering major in the Electrical Engineering and Computer Science Department at the University of Arkansas. He is an international exchange student from the University Charles III of Madrid.

Dr. David C. Jensen - Dr. Jensen is an Assistant Professor in the Department of Mechanical Engineering at the University of Arkansas. His research focuses on using models to understand the effects of and detect failures in systems and has previously worked on projects that use machine learning algorithms to detect faults in rotary machines.

9.0 Facilities and Equipment

Our team utilized the SpectraQuest Machinery Fault Simulator located in Dr. Jensen's lab at the ENRC. The machine has several known faulty bearings and warped shafts to simulate real world issues that our program will help detect and analyze while the machine is running. We were given full access to the facility to use the machine and run it and the software it uses to collect data so that we could better understand how the machine functions.

10.0 References

- [1] SpectraQuest Inc., "Machinery Fault Simulator," *SpectraQuest*, accessed Oct. 1, 2024.
- [2] Oxford University Press, "Machine Learning – Definition," *Oxford Advanced Learner's Dictionary*, accessed Oct. 1, 2024.
- [3] IBM, "What Is Machine Learning (ML)?" *IBM*, Sep. 5, 2024.
- [4] D. K. Saha *et al.*, "Development of intelligent fault diagnosis technique of rotary machine element bearing: A machine learning approach," *Sensors*, vol. 22, no. 3, p. 1073, Jan. 2022.
- [5] L. Marshall and D. Jensen, "Dataset of single and double faults scenarios using vibration signals from a rotary machine," *Data in Brief*, Jul. 4, 2023.
- [6] L. Marshall Jr., D. Jensen, and H. Hu, "Supporting condition-based maintenance for rotary systems under multiple fault scenarios," in *Proc. ASME Int. Design Eng. Tech. Conf. & Comput. Inf. Eng. Conf.*, vol. 2, Boston, MA, USA, Aug. 2023, Paper no. DETC2023-116470.
- [7] A. K. S. Jardine, D. Lin, and D. Banjevic, "A review on machinery diagnostics and prognostics implementing condition-based maintenance," *Mech. Syst. Signal Process.*, vol. 20, no. 7, pp. 1483–1510, 2006.
- [8] MathWorks, "FFT – Practical introduction to frequency domain analysis," *MATLAB & Simulink*, accessed Nov. 13, 2024.
- [9] IBM, "What is Support Vector Machine?" *IBM*, Aug. 13, 2024.
- [10] J. Xiong, "An introduction to statistical machine learning: The crucial role of statistics in modern technology," *DataCamp*, Jun. 30, 2023.
- [11] A. A. Awan, "What are neural networks?" *DataCamp*, Aug. 30, 2023.

- [12] Google Developers, “Classification: Accuracy, recall, precision, and related metrics,” *Machine Learning Crash Course*, accessed Nov. 13, 2024.
- [13] J. Jiao, H. Li, and S. Zhang, “Fault diagnosis of rolling element bearings using time-frequency domain features and wavelet transform,” *Mech. Syst. Signal Process.*, vol. 198, 2023.
- [14] T. Huynh and H. Min, “Vibration signal-based machine learning classification for fault diagnosis in rotating machinery,” *IEEE Trans. Ind. Electron.*, vol. 71, no. 1, pp. 56–67, Jan. 2024.
- [15] X. Sun, B. Li, and Y. Chen, “Bearing remaining useful life prediction using a CNN-Transformer hybrid network,” *IEEE Access*, vol. 11, pp. 15678–15690, 2023.
- [16] Y. Guo, D. Wang, and J. Yu, “A review on data-driven bearing prognostics using artificial intelligence techniques,” *Mech. Syst. Signal Process.*, vol. 105, pp. 1–37, 2018.
- [17] S. Singh and S. Mohanty, “Gearbox fault detection using envelope analysis and statistical features,” *J. Vib. Control*, vol. 28, no. 6, pp. 712–725, 2022.
- [18] R. K. Mobley, *An Introduction to Predictive Maintenance*, 2nd ed. Burlington, MA, USA: Butterworth-Heinemann, 2002.
- [19] A. Widodo and B.-S. Yang, “Support vector machine in machine condition monitoring and fault diagnosis,” *Mech. Syst. Signal Process.*, vol. 21, no. 6, pp. 2560–2574, Aug. 2007.
- [20] A. Jardine, D. Lin, and D. Banjevic, “A review on machinery diagnostics and prognostics implementing condition-based maintenance,” *Mech. Syst. Signal Process.*, vol. 20, no. 7, pp. 1483–1510, 2006.